

ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS

PROYECTO FINAL
VALIDACIÓN Y VERIFICACIÓN DE SOFTWARE

Hospital Management System

Informe Final de Entregables

Integrantes: Erick Costa – Líder Backend
Jimmy Arias – Líder Frontend
Paul Salas – Líder QA, Calidad y Seguridad

Profesor: Christian Suárez

Asignatura: Validación y Verificación de Software

Período Académico: 2026A

Fecha: 19 de julio de 2026

Repositorio: <https://github.com/zeuspyEC/hospital-management>

Índice

1. Introducción	2
2. Distribución del equipo	2
3. Arquitectura del sistema	3
4. Flujo de trabajo y control de versiones	3
5. Pruebas Unitarias – Backend	4
6. Pruebas de Integración – Backend	4
7. Pruebas Unitarias – Frontend	6
8. Pruebas E2E – Frontend (Playwright)	6
9. Análisis Estático de Código	8
10. Análisis de Seguridad – OWASP Top 10	8
11. Resumen consolidado de ejecución	9
12. Bugs reales encontrados no documentados en el enunciado	9
13. Estructura de entregables en el repositorio	9
14. Reproducibilidad	10

1. Introducción

Este documento consolida los entregables del proyecto final de Validación y Verificación de Software sobre el *Hospital Management System*: un sistema fullstack (Spring Boot 3.3 + JavaScript vanilla + PostgreSQL 16) provisto como base por la cátedra, con deficiencias intencionales que el grupo debía detectar mediante pruebas sistemáticas, análisis estático y análisis de seguridad.

Todo resultado presentado aquí proviene de **ejecuciones reales** de las herramientas correspondientes contra el código del repositorio, no de estimaciones. Cada sección referencia el comando exacto usado y el archivo de reporte crudo correspondiente en `docs/informes/` o `informes/`, para que puedan reproducirse de forma independiente.

Este informe describe **qué se construyó y qué resultado dio al ejecutarlo**; la evaluación y calificación de cada entregable frente a la rúbrica del curso corresponde al profesor.

2. Distribución del equipo

Integrante	Rol	Áreas de responsabilidad	Rama Git
Erick Costa	Líder Backend	Configuración del backend, pruebas unitarias (JUnit 5 + Mockito), pruebas de integración (SpringBootTest + MockMvc + H2), corrección de un bug real de serialización que rompía <code>/api/citas</code> y <code>/api/historias-clinicas</code> .	<code>feature/backend-erick</code>
Jimmy Arias	Líder Frontend	Pruebas unitarias con Jest (<code>utils.js</code> , <code>api.js</code>), pruebas End-to-End con Playwright sobre los 4 flujos principales.	<code>feature/frontend-jimmy</code>
Paul Salas	Líder QA, Calidad y Seguridad	Análisis estático (Checkstyle, SpotBugs + FindSecBugs, ESLint), análisis de seguridad OWASP Top 10, documentación de hallazgos.	<code>feature/qa-paul</code>

El flujo de ramas (`main` ← `develop` ← `feature/*`), las reglas de commit y el checklist de Pull Request usados por el equipo están definidos en `.claude/skills/team-git-workflow/SKILL.md` y `CONTRIBUTING.md`, ambos versionados en el repositorio.

3. Arquitectura del sistema

- **Backend:** Spring Boot 3.3.0, Spring Data JPA, Jakarta Validation, arquitectura en capas (**Controller** → **Service** → **Repository**), con DTOs para la comunicación REST y un **GlobalExceptionHandler** centralizado.
- **Frontend:** JavaScript vanilla modular (**app.js**, **api.js**, **utils.js** y un módulo por entidad), sin bundler ni framework – los objetos de cada módulo se referencian desde atributos **onclick** en HTML generado dinámicamente.
- **Base de datos:** PostgreSQL 16 en contenedor Docker, con **schema.sql/data.sql** de inicialización.
- **Entidades:** Paciente, Doctor, Cita (relación **@ManyToOne** a Doctor, pero **pacienteId** es solo una columna sin FK) e HistoriaClinica (relaciones a Paciente y Doctor).

4. Flujo de trabajo y control de versiones

Rama	Commits desde main	Contenido principal
feature/backend-erick	7	4 clases de pruebas unitarias de servicio, 4 clases de integración de controlador, configuración de JaCoCo, corrección del bug de serialización Jackson/Hibernate.
feature/frontend-jimmy	8	utils.test.js , api.test.js , 4 specs de Playwright, configuración de Jest/Playwright.
feature/qa-paul	10	Configuración de Checkstyle/SpotBugs/ESLint, reportes crudos, analisis-estatico.pdf , analisis-owasp.pdf .

Todas las ramas se integran hacia **develop** mediante merge tras verificar que la suite de pruebas correspondiente pasa en verde; el historial completo es público en el repositorio indicado en la carátula.

5. Pruebas Unitarias – Backend

Herramientas: JUnit 5 + Mockito. **Ubicación:** backend/src/test/java/com/hospital/service/.

Clase	Casos felices	Casos límite	Errores	Total
PacienteServiceTest	5	3	4	12
DoctorServiceTest	5	3	2	10
CitaServiceTest	5	3	3	11
HistoriaClinicaServiceTest	5	2	3	10
Total	20	11	12	43

Salida real de mvn test (JDK 17, 19 de julio de 2026):

```
[INFO] Tests run: 43, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Cada clase usa mocks de sus repositorios (@Mock + @ExtendWith(MockitoExtension.class)) y está organizada en tres grupos @Nested: CasosFelices, CasosLimite y ManejoDeErrores. Varios de los casos límite reproducen explícitamente bugs intencionales del código base (división por cero en calcularEdadPromedio, doble booking en citas, inyección SQL en buscarPorEspecialidadInsegura, XSS almacenado sin sanitizar en historias clínicas).

6. Pruebas de Integración – Backend

Herramientas: @SpringBootTest + MockMvc + H2 en memoria. **Ubicación:** backend/src/test/java/com

Clase	Tests	Cobertura funcional
PacienteControllerIntegrationTest	9	GET/POST/PUT/DELETE, validación, búsqueda
DoctorControllerIntegrationTest	9	GET/POST/PUT/DELETE, inyección SQL reproducida vía HTTP
CitaControllerIntegrationTest	11	GET/POST/PUT/DELETE, doble booking, rango de fechas
HistoriaClinicaControllerIntegrationTest	9	GET/POST, XSS almacenado, validación
Total (unitarias + integración)	81	

Salida real de mvn test (suite completa):

```
[INFO] Tests run: 81, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Cobertura de código (JaCoCo, generado con mvn test jacoco:report):

Paquete	Instrucciones	Ramas
com.hospital.service	97 %	64 %
com.hospital.controller	93 %	n/a
com.hospital.model	98 %	n/a

Paquete	Instrucciones	Ramas
com.hospital.dto	100 %	n/a
com.hospital.config	100 %	n/a
com.hospital.exception	74 %	100 %
Total del proyecto	95 %	68 %

Reporte HTML completo disponible localmente en `backend/target/site/jacoco/index.html` tras correr `mvn test` (no versionado, es un artefacto de build).

Bug real corregido durante esta fase (no intencional, no documentado en comentarios del código): `GET /api/citas` y `GET /api/historias-clinicas` devolvían 500 de forma consistente contra PostgreSQL real, por un `InvalidDefinitionException` de Jackson al serializar proxies Hibernate de relaciones `@ManyToOne(LAZY)`. Se agregó el módulo `jackson-datatype-hibernate6` con `FORCE_LAZY_LOADING` (para no ocultar el bug intencional de consultas N+1 que el enunciado pide detectar). Detalle completo en el commit `fix: corrige 500 en GET /api/citas...` de `feature/backend-erick`.

7. Pruebas Unitarias – Frontend

Herramientas: Jest 29 + jsdom. **Ubicación:** frontend/js/__tests__/.

Archivo	Tests	Cobertura	Alcance
utils.test.js	32	100 % sentencias	Las 8 funciones exportadas de utils.js
api.test.js	17	55 % sentencias	apiFetch + muestra de los 4 módulos de API (fetch mockeado)
Total	49		

Salida real de `npx jest -coverage`:

```
Test Suites: 2 passed, 2 total
Tests: 49 passed, 49 total
utils.js | 100 | 90.9 | 100 | 100
```

Para poder importar `utils.js` y `api.js` desde Jest (son scripts globales sin `module.exports`, pensados para cargarse con `<script>` en el navegador) se agregó un bloque `if (typeof module !== 'undefined') module.exports = {...}` al final de cada archivo, sin modificar ninguna función existente.

Bugs reales encontrados al ejecutar los tests (contradicen los comentarios "BUG INTENCIONAL" del propio código fuente):

- **apiFetch pierde el header Content-Type** cuando se le pasan headers personalizados, porque `{headers: {...}, ...options}` hace que `options.headers` sobrescriba por completo el objeto ya fusionado. No estaba documentado.
- El regex de `validateEmail` **sí rechaza** `usuario@dominio` y **sí acepta** emails con `+`, exactamente lo contrario de lo que afirma el comentario en el código fuente.

8. Pruebas E2E – Frontend (Playwright)

Herramientas: Playwright 1.47 (Chromium). **Ubicación:** frontend/e2e/. Ejecutadas contra el **stack real completo** (PostgreSQL en Docker, backend Spring Boot en :8080, frontend estático en :3000), no contra mocks.

Archivo	Tests	Flujo cubierto
pacientes.spec.js	3	CRUD completo, validación HTML5, búsqueda
doctores.spec.js	3	CRUD completo, especialidad opcional (bug), búsqueda
citas.spec.js	4	Creación/edición, doble booking (bug), filtro por estado
historias.spec.js	3	Creación + consulta con "Ver", doctor opcional, XSS almacenado (bug)
Total	13	

Salida real de `npx playwright test`:

```
Running 13 tests using 1 worker
13 passed (10.3s)
```

Reporte HTML con capturas (generado automáticamente en cada falla, y disponible siempre en `frontend/playwright-report/` tras correr la suite) mediante `npx playwright show-report`.

Hallazgo relevante encontrado durante la ejecución: el DELETE de pacientes y doctores **sí elimina el registro en el backend** (200 con cuerpo vacío), pero `apiFetch` intenta parsear ese cuerpo vacío como JSON y lanza una excepción; la UI muestra `.Error` al eliminar **no refresca la tabla**, dejando una fila fantasma visible aunque el registro ya no exista en la base de datos. Confirmado recargando la página dentro del mismo test.

9. Análisis Estático de Código

Herramientas configuradas y ejecutadas: **Checkstyle** (Google Java Style), **SpotBugs** + **FindSecBugs** (`effort=Max, threshold=Low`) y **ESLint** (`eslint:recommended`).

Herramienta	Hallazgos	Más frecuente	Reporte
Checkstyle	937	Indentación (553)	<code>checkstyle-report.txt</code>
SpotBugs + FindSecBugs	37	SPRING_ENDPOINT (28, informativo)	<code>spotbugs-report.txt</code>
ESLint	4 (warnings)	no-unused-vars (falso positivo estructural)	<code>eslint-report.txt</code>

El informe completo, con la clasificación por severidad, el análisis detallado de los hallazgos más relevantes (incluyendo un **falso negativo confirmado**: FindSecBugs no detectó la inyección SQL real de **DoctorService**) y la configuración exacta de cada plugin, está en [informes/analisis-estatico.pdf](#). Los reportes crudos (`.txt/.xml/.json`) están en `docs/informes/`.

10. Análisis de Seguridad – OWASP Top 10

Se identificaron **11 hallazgos**, todos verificados de forma reproducible (petición HTTP real contra el backend en ejecución, o test automatizado en verde), mapeados a 6 categorías del OWASP Top 10 (2021):

#	Hallazgo	Categoría	Riesgo
1	Ausencia total de autenticación/autorización	A01	Alto
2	CORS permisivo (<code>origins="*"</code>)	A01	Medio
3	Credenciales de BD en texto plano en el repositorio	A02	Alto
4	Inyección SQL real (búsqueda de doctores)	A03	Alto
5	XSS almacenado en historias clínicas	A03	Alto
6	XSS por <code>innerHTML</code> en Pacientes/Doctores/Citas	A03	Medio
7	XSS en <code>showAlert()</code> (compartido por todos los módulos)	A03	Medio
8	Componentes desactualizados (Spring Boot 3.3.0 → 4.1.0 disponible)	A06	Medio
9	Códigos HTTP incorrectos (200 en vez de 404/201/204)	A05	Medio
10	Exposición de stack traces completos en errores 500	A05	Medio
11	Sin logging del servidor ante errores no controlados	A09	Alto

Cada hallazgo incluye, en el informe completo, ubicación exacta (archivo:línea), impacto, evidencia de reproducción y mitigación recomendada: [informes/analisis-owasp.pdf](#).

11. Resumen consolidado de ejecución

Suite	Tests	Resultado	Comando
Unitarias backend (JUnit 5 + Mockito)	43	0 fallos	<code>mvn test</code> <code>-Dtest=*ServiceTest</code>
Integración backend (SpringBootTest + H2)	38 (81 total con unitarias)	0 fallos	<code>mvn test</code> <code>-Dtest=*ControllerIntegrationT</code>
Unitarias frontend (Jest)	49	0 fallos	<code>npx jest</code>
E2E frontend (Playwright)	13	0 fallos	<code>npx playwright</code> <code>test</code>
Total pruebas automatizadas	143	0 fallos	

12. Bugs reales encontrados no documentados en el enunciado

Además de los bugs intencionales del enunciado (que las pruebas de las secciones anteriores reproducen sistemáticamente), el equipo encontró y documentó los siguientes defectos **no anunciados** en los comentarios del código base:

1. `GET /api/citas` y `GET /api/historias-clinicas` rompían por completo (500) contra PostgreSQL real – corregido (ver Sección 5).
2. `apiFetch` pierde el header `Content-Type` al recibir headers personalizados (bug de JavaScript, no mencionado en los comentarios de `api.js`).
3. El comentario "BUG INTENCIONAL" de `validateEmail` en `utils.js` describe un comportamiento que el regex real no tiene (verificado con Node.js y con tests).
4. El comentario "BUG INTENCIONAL" de `PacienteRepository.buscarPorNombre` afirma que la query es vulnerable a inyección SQL, pero usa un parámetro con *binding* (`:nombre`) correctamente parametrizado – no es explotable, a diferencia del método de `DoctorService` que sí lo es.
5. El DELETE de pacientes/doctores deja una fila fantasma en la UI por el manejo incorrecto de respuestas con cuerpo vacío en `apiFetch` (ver Sección 8).

13. Estructura de entregables en el repositorio

```

hospital-management/
|-- backend/src/test/java/com/hospital/
| |-- service/ (4 clases, 43 tests)
| '-- controller/ (4 clases, integracion con H2)
|-- frontend/
| |-- js/__tests__/ (utils.test.js, api.test.js)
| '-- e2e/ (4 specs de Playwright)
|-- informes/
| |-- analisis-estatico.pdf
| '-- analisis-owasp.pdf
|-- docs/
| |-- informes/ (reportes crudos: checkstyle, spotbugs, eslint)
| '-- informe-final/ (este documento)
|-- .claude/skills/team-git-workflow/SKILL.md
|-- CONTRIBUTING.md
'-- .github/PULL_REQUEST_TEMPLATE.md

```

14. Reproducibilidad

Todos los resultados de este informe pueden reproducirse con:

```
# Base de datos
docker compose up -d

# Backend (requiere JDK 17)
cd backend && mvn test # 81 tests
mvn spotbugs:spotbugs && mvn checkstyle:check

# Frontend
cd frontend && npm install
npx jest --coverage # 49 tests
npx eslint "js/**/*.js" "e2e/**/*.js"

# E2E (requiere backend + frontend + DB arriba)
npx playwright install chromium
npx playwright test # 13 tests
```